

Column Generation & Branch-and-Price

Lab deck – installation, running, exercises

Antoine Legrain

May 20, 2026

Polytechnique Montréal

What this deck contains

- Setup: Python virtual environment, dependencies, smoke check.
- File map of the `python/` folder.
- How to run the smoke check on `toy.txt`.
- Exercises A through E, in execution order, with each `EX-X.k` hole listed explicitly so nothing is skipped.
- Troubleshooting recipes.
- Cheat sheet: HiGHS & `rcspp` APIs.

Open this deck (on a second screen) *while you code*. The theory is in `theory.pdf`.

1. Installation

What you need on your laptop

- Python ≥ 3.10 recommended (works on ≥ 3.8).
- `pip` and `venv` (standard library on most systems).
- **Windows users:** use WSL, Git Bash, or Anaconda – `venv` works the same way once Python is on `PATH`.
- Network access for the first install: HiGHS / NumPy / NetworkX / matplotlib come from PyPI, `rcspp` from TestPyPI.
- About 200 MB of disk for the `venv`.

macOS / Homebrew

If your system has multiple Pythons, prefer the Homebrew one (`brew install python`) over the one shipped by Apple.

Create the virtual environment

Linux / macOS: `cd python/ && python3 -m venv .venv && source .venv/bin/activate`

Windows PowerShell: `cd python/ ; python -m venv .venv ; .venv\Scripts\activate`

Then, in both cases:

```
(.venv) $ python -m pip install --upgrade pip
(.venv) $ pip install -r requirements.txt
(.venv) $ pip install --index-url https://test.pypi.org/simple/ \
               --extra-index-url https://pypi.org/simple \
               rcspp==0.0.1.dev0
```

Always activate the venv

Re-activate the venv in every new terminal, otherwise `import highspy` or `import rcspp` picks up the system Python.

Running against the reference solution

Once your environment is healthy you can run any of the same checks using the reference implementation under `solutions/`:

```
(.venv) $ cd solutions/  
(.venv) $ python -m vrp.smoke_test  
(.venv) $ python -m vrp.tests.exA_master  
(.venv) $ python -m vrp.tests.exB_pricing  
(.venv) $ python -m vrp.tests.exC_cg  
(.venv) $ python -m vrp.tests.exD_diving  
(.venv) $ python -m vrp.tests.exE_bnp
```

Each check defaults to the small `toy.txt` instance; pass `--instance R101_25.txt` for a larger Solomon file.

Smoke check – you should see this

```
(.venv) $ python -m vrp.smoke_test
[smoke] highspy version: 1.10.0
[smoke] rcspg version: 0.0.1
[smoke] toy instance: 7 customers, capacity 50
[smoke] rcspg solve: 1 sink label, cost = -2.00
[smoke] OK -- environment is ready.
```

If you see *any* other line marked [smoke] ERROR, stop and fix the install before going further – the exercises will not work otherwise.

Reproducing the smoke check

The smoke check does not touch any code you'll modify, so you can re-run it at any time to confirm your environment is still healthy.

2. Project tour

The python/ folder

```
python/
+-- README.md, requirements.txt, theory.pdf, lab.pdf
+-- instances/          toy.txt + Solomon files (+ duals/<inst>/iter_*.txt)
+-- solutions/          reference solutions (do not peek!)
+-- vrp/
    +-- instance.py, instance_reader.py    (read-only)
    +-- smoke_test.py                     environment check
    +-- log.py                             vrp.log configure / get_logger
    +-- plot.py                            CG / B&P convergence figures
    +-- vrp.py                             HOLES: A.3, B.3, C.2-C.4
    +-- cg/
        | +-- master_problem.py            HOLES: A.1, A.2, D.1
        | +-- pricing.py                   HOLES: B.1, B.2, E.4
        | +-- bound.py                     HOLE : C.5
        | +-- diving.py                    HOLES: D.2, D.3
        | +-- bnp.py                       HOLES: E.1, E.2, E.3 (loop is provided)
        | +-- mp_solution.py, path.py      (read-only)
    +-- tests/                             progressive checks (NOT pytest)
        +-- exA_master.py, exB_pricing.py, exC_cg.py,
        +-- exD_diving.py, exE_bnp.py,
        +-- compare_cg.py, compare_bnp.py  parametric studies + matplotlib
```

The TODO holes – complete checklist

Tag	File	What to fill in
EX-A.1	cg/master_problem.py	vehicle constraint $\sum_p x_p \leq K$, plus one = 1 row per customer
EX-A.2	cg/master_problem.py	add one column per path (no upper bound!)
EX-A.3	vrp.py	generate one round-trip per customer
EX-B.1	cg/pricing.py	build the resource graph (nodes + arcs)
EX-B.2	cg/pricing.py	per-arc base cost & dual Row
EX-B.3	vrp.py	wire solve_subproblem (build once + update_reduced_costs)
EX-C.1	vrp.py	multi-column add to master
EX-C.2	vrp.py	Wentges dual smoothing (optional)
EX-C.3	vrp.py	per-iteration log line (LP, LB, UB, gap)
EX-C.4	cg/bound.py	Lagrangian bound formula
EX-D.1	cg/master_problem.py	implement _set_integrality (RMH)
EX-D.2	cg/diving.py	pick the column to fix
EX-D.3	cg/diving.py	re-run CG after a fix
EX-E.1	cg/bnp.py	aggregated arc flow $\bar{y}_{uv}$
EX-E.2	cg/bnp.py	most-fractional arc
EX-E.3	cg/bnp.py	build the two children
EX-E.4	cg/pricing.py	skip forbidden arcs

The B&P search loop itself is **already coded** in `bnp.py`.

3. Exercise A – Master in HiGHS

Exercise A – goal

Exercise A – master & initial columns

Holes to fill: EX-A.1, EX-A.2, EX-A.3.

- **A.1:** in `cg/master_problem.py`, build the **vehicle-count constraint** $\sum_p x_p \leq K$ and one =1 covering row per customer. (The vehicle row gives the dual σ that the Lagrangian bound needs.)
- **A.2:** in the same file, add a column per Path. Bounds: `lower=0`, **no upper bound** (`upper = highs.py.kHighsInf`). The column has coefficient 1 on the vehicle row.
- **A.3:** in `vrp.py`, complete `generate_initial_paths`: one round-trip $0 \rightarrow i \rightarrow 0$ per customer.

Run the check

```
(.venv) $ python -m vrp.tests.exA_master
```

Exercise A – expected output

```
[ex.A] instance: toy.txt (6 customers, K=6 vehicles)
[ex.A] building master with 6 initial columns
[ex.A] LP optimum (with slacks): 228.30
[ex.A] vehicle dual sigma      : -0.00
[ex.A] customer duals:
[ex.A]     pi_1=42.43  pi_2=41.23  pi_3=22.36
[ex.A]     pi_4=41.23  pi_5=50.00  pi_6=31.05
[ex.A] OK -- master is feasible and returns finite duals.
```

Why no upper bound on x_p ?

The covering constraint $\sum_p a_{ip} x_p = 1$ already bounds x_p from above whenever $a_{ip} \geq 1$. Adding $x_p \leq 1$ would create an extra dual μ_p that the pricer would have to subtract from $\bar{c}_p$. We never do that.

4. Exercise B – Pricing wiring

Exercise B – arc costs & RCSPP solve

Holes to fill: EX-B.1, EX-B.2, EX-B.3.

- B.1: in `cg/pricing.py`, build the graph *once* – 3 resources, all customer nodes, a sink, and all arcs. No duals yet.
- B.2: per arc, push the **base** resource increments (d_{uv} , $s_u + d_{uv}$, q_v) and a dual `Row(origin_id, 1.0)` so that the reduced cost becomes $\bar{c}_{uv} = d_{uv} - \pi_u$ after `update_reduced_costs`.
- B.3: in `vrp.py`, wire `solve_subproblem(duals)`: lazily build the graph on the first call, then `update_reduced_costs(duals)` and return `graph.solve()` each iteration.

```
(.venv) $ python -m vrp.tests.exB_pricing
```

Exercise B – expected output

```
[ex.B] loading saved duals from instances/duals/R101_25/iter_0.txt
[ex.B] best route          : [0, 14, 15, 3, 24, 25, 26]
[ex.B] reduced cost        : -138.98
[ex.B] # neg-cost routes : 58
[ex.B] OK
```

Common bug

Forgetting that the depot's dual is 0 leaves a phantom $-\infty$ on every depot-to-depot arc. Initialise $\pi[0] = \sigma$ explicitly.

5. Exercise C – Efficient CG

Exercise C – goal

Exercise C – multi-column, smoothing, LB

Holes to fill in `vrp.py`,: **EX-C.1, EX-C.2, EX-C.3, EX-C.4.**

- **C.2:** cap pricer output at `K_MAX` columns; only those with $\bar{c} < -\epsilon$ are added to the master.
- **C.3:** Wentges smoothing $\pi \leftarrow \alpha \pi^{prev} + (1 - \alpha) \pi$.
- **C.4:** per-iteration log line `iter`, `n_cols`, `LP`, `LB`, `UB`, `gap`, plus the gap-stop (`gap_pct`) and UB-cut (`incumbent_ub`) early exits already wired in `solve_cg`.
- **C.5:** in `cg/bound.py`, implement $LB = \sum_i \pi_i + K\sigma + K \cdot \min(0, \bar{c}^*)$ (Trick 7 in the theory deck).

Try `--K 152050` and `-a 00.30.5` (or `compare_cg` below) to feel each knob.

Don't rebuild the master or the graph

Build both *once* in Exercise A/B. Each CG iteration calls `master.add_column(...)` on the new columns and `graph.update_reduced_costs(...)` – both warm-start.

Exercise C – expected output (R101_25, K=25)

```
(.venv) $ python -m vrp.tests.exC_cg
[cg] iter cols LP LB gap%
[cg] 0 75 1246.16 -2228.39 155.9%
[cg] 1 90 703.94 -447.45 257.3%
[cg] 2 95 674.01 8.45 7877%
[cg] 3 100 665.92 283.68 134.7%
[cg] 4 104 640.45 444.00 44.2%
[cg] 5 110 640.45 375.12 70.7%
[cg] 6 114 637.33 547.37 16.4%
[cg] 7 114 636.28 636.28 0.0%
[cg] CG done in 8 iters, LP=636.28, LB=636.28, 89 cols added
```

Try `--nb-cols` $\in \{1, 5, 20, 100\}$ and observe the iteration count drop. Without `--nb-vehicles 25` the first iterations show slack-inflated values.

6. Exercise D – Heuristic IP

Exercise D – RMH and plain diving

Holes to fill: EX-D.1, EX-D.2, EX-D.3.

- **D.1:** in `cg/master_problem.py`, complete `_set_integrality(integer)`; **this is required** for the RMH to actually solve a binary MIP. Without it, `master.solve(relax=False)` silently returns the LP solution.
- **D.2, D.3:** in `cg/diving.py`, pick the column with the largest fractional value, fix it to 1, then re-run CG. `plain_dive` also takes an optional `ub`: when the LP at a dive node already exceeds `ub`, the dive aborts (saves work inside B&P).

```
(.venv) $ python -m vrp.tests.exD_diving
```

Exercise D – expected output

```
[ex.D] root LP=636.3
[ex.D] RMH incumbent (UB)=636.3, time=0.1s
[ex.D] RMH solution:
[ex.D]   cost=81.11 [0, 11, 19, 10, 26]
[ex.D]   cost=59.67 [0, 7, 8, 26]
[ex.D]   cost=46.47 [0, 2, 21, 26]
... (9 routes, total 636.28)
[diving] dive start: LP=636.28
[diving] dive end (integer): 0 fixes, UB=636.28
[ex.D] summary: RMH UB=636.3   dive UB=636.3   LP=636.3
```

LP already integer on R101_25

The CG LP is integer for this instance – RMH solves the exact MIP and the dive terminates with 0 fixes. On harder instances (e.g. C101_25.txt or RC101.txt) you will see actual fixes and a gap between LP and RMH. Decrease the number of vehicles with `--nb-vehicles` to make the instance harder.

7. Exercise E – Branch-and-Price

Exercise E – arc-branching B&P

Holes to fill in `cg/bnp.py`: **EX-E.1, E.2, E.3, E.4.**

- **E.1:** aggregated arc flow $\bar{y}_{uv}$.
- **E.2:** most-fractional arc.
- **E.3:** two children – forbid / force. The *force* branch uses `_up_compatible(p,u,v)`: it drops every path that leaves u to some $w \neq v$ or arrives at v from $w \neq u$.
- **E.4:** in `pricing.py`, skip forbidden arcs at graph-build time.

The B&P search loop is **already coded** (`cg/bnp.py`) and returns a `BnPIncumbent` carrying cost, `sol` and the `{ub,lb,time}_history` traces used by `plot_bnp_progress`. The Lagrangian bound was implemented in Exercise C (Trick 7). Dive at root, RMH every `--rmh-every` nodes; `--gap` stops within $\delta\%$; `--no-depth-first` switches to pure best-first.

Exercise E – expected output

```
(.venv) $ python -m vrp.tests.exE_bnp
[bnp] B&P start: root LB=1636.39, 1370 columns in pool, depth_first=True ...
[bnp] B&P done in 12 nodes, optimum=1642.88 (20 routes), global LB=1642.88 ...
[ex.E] done in 5.9s, optimum=1642.88
[ex.E] solution (20 routes):
[ex.E]   cost=86.77  [0, 92, 42, 15, 87, 57, 97, 101]
[ex.E]   cost=50.00  [0, 31, 88, 7, 101]
[ex.E]   cost=107.35 [0, 63, 64, 49, 48, 101] ...
```

1 node = proven optimal without branching

R101_25 has an integer root LP (you can try) – the B&P terminates immediately. On larger or more constrained instances (RC101.txt here) the tree has many nodes and branching is essential. Use `compare_bnp` to observe the search progress.

8. Comparison runners

Two scripts to compare configurations

After exercises C and E you have working CG and B&P. The lab kit ships two helpers that re-run them with different knobs and plot the convergence side-by-side (vrp.plot uses matplotlib):

```
# CG -- compare pricing batch sizes
python -m vrp.tests.compare_cg --instance RC101.txt --Ks 1 5 20 50
# CG -- compare Wentges smoothing
python -m vrp.tests.compare_cg --instance RC101.txt --alphas 0 0.2
    0.5
# B&P -- compare gap thresholds
python -m vrp.tests.compare_bnp --instance R101.txt --gaps 0 0.5 2
# B&P -- depth-first vs best-first
python -m vrp.tests.compare_bnp --instance R101.txt --search both
```

Add `--save figure.png` to write the figure to disk instead of displaying it.

What the figures show

- `plot_cg_convergence` – LP & LB per iteration, plus the duality gap, one curve per run. Reads `CGStats.lp_history` and `CGStats.lb_history`.
- `plot_bnp_progress` – UB / LB and gap per node *and* per wall-clock second, one curve per run. Reads `BnPIncumbent.{ub,lb,time}_history`.

Both `CGStats` and `BnPIncumbent` record their history as the algorithm progresses – no extra wiring needed.

Use the runners for the lab write-up

Pick one figure that surprises you (e.g. where best-first beats depth-first, or where Wentges $\alpha=0.5$ wins on iterations but loses on time) and add a paragraph of analysis to your hand-in.

9. Troubleshooting

Five errors that always show up

Symptom	Cause	Fix
ImportError: highspy	venv not active	source .venv/bin/activate
LP value = 0 at iter 0	forgot $b_i = 1$ on covering rows	set RHS=1 in EX-A.1
pricer returns empty	π_0 not zeroed	force $\pi[0]=0$ before pricing
LB > LP early on	$\bar{c}^*$ from heuristic pricer	LB only valid with exact pricer
B&P loops forever	forgetting to filter parent pool	drop incompatible columns at child node

Get unstuck fast

Each saved `iter_*.txt` in `instances/duals/R101/` is a one-iteration regression check. Diff your duals against them.

10. Cheat sheet

HiGHS in Python – the calls you need

```
import highspy as hp
m = hp.Highs()
m.silent()                                # no chatter

# row: equality constraint sum a_ip * x_p == 1
m.addRow(1.0, 1.0, 0, [], [])            # empty row, fill later
# column: cost = c_p, lb=0, ub=+inf (no upper bound!)
m.addCol(c_p, 0.0, hp.kHighsInf,
         len(idx), idx, val)              # idx = row indices

m.run()                                   # solve LP
duals = list(m.getSolution().row_dual)    # row duals = pi
sol    = list(m.getSolution().col_value)  # primal x_p
m.changeColIntegrality(j, hp.HighsVarType.kInteger) # for MIP / RMH
m.changeColBounds(j, 1.0, 1.0)            # fix column to 1 (B&P / dive)
```



```
from rcspp.graph      import ResourceGraph
from rcspp._core.graph import Row
from rcspp.resource   import (
    AdditionExtensionFunction, ValueDominanceFunction,
    ValueCostFunction, TrivialFeasibilityFunction,
    MinMaxFeasibilityFunction,
    TimeWindowExtensionFunction, TimeWindowFeasibilityFunction,
)

g = ResourceGraph()
g.add_real_resource(ext, feas, cost, dom)           # once per resource
g.add_node(node_id, is_source, is_sink)
g.add_arc((dist, time, load), o, d, arc_id, dist,   # base costs
          [Row(o, 1.0)])                           # subtract pi_o

g.update_reduced_costs(dual_by_id)                 # cheap re-pricing
sols = g.solve()                                   # list[Solution]
# .cost, .path_node_ids, .path_arc_ids
```

Build the graph once (in `PricingGraph.build`); each CG iteration calls only `update_reduced_costs(dual_by_id)`.

- Bonus exercise **F**: replace plain dominance by **ng-routes** in `pricing.py` and measure the speed-up on `R101_50.txt`.

Have fun, and break the LP, not the dominance!